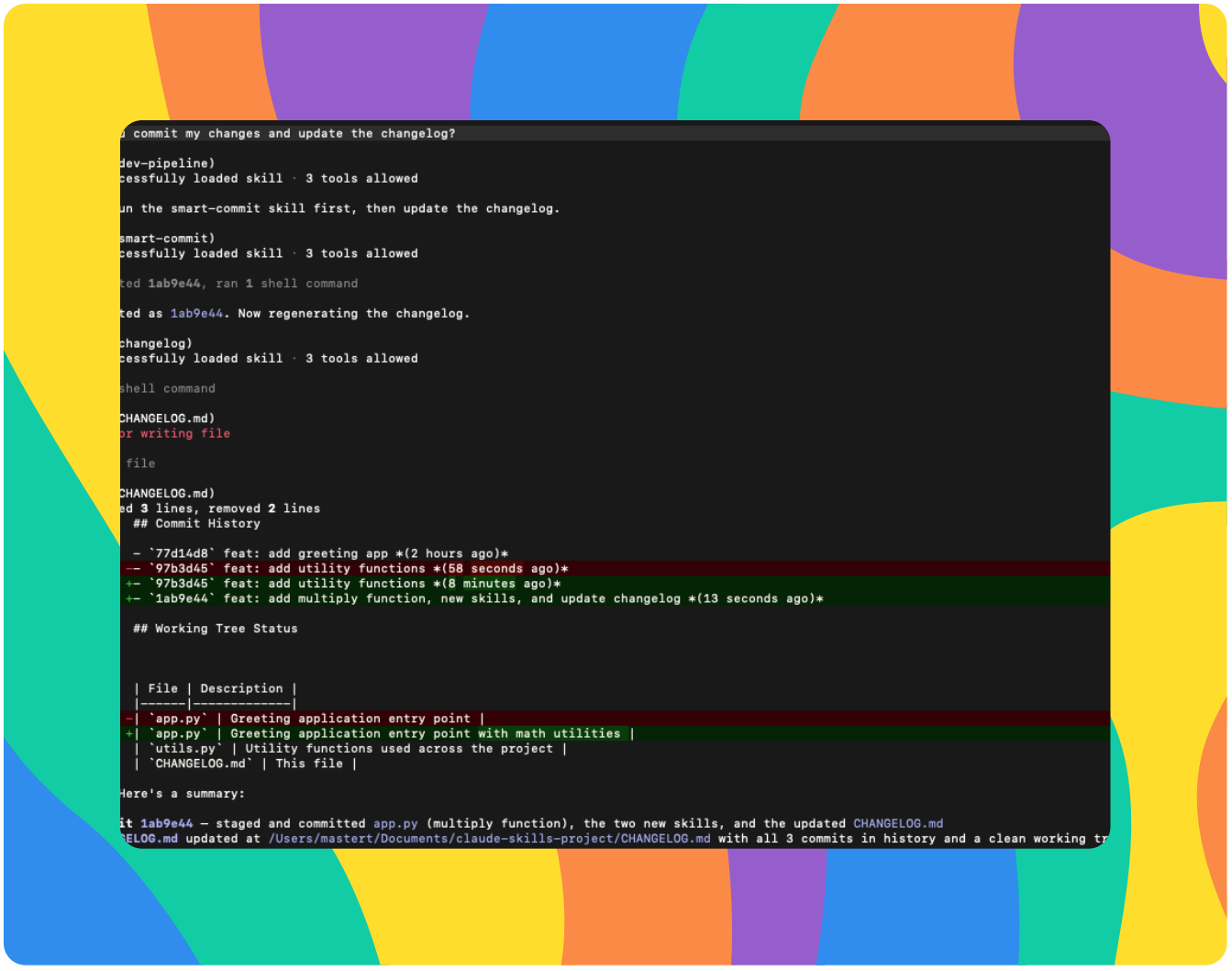
Claude Code Skills Basics

TA

tahirgroot@gmail.com
https://medium.com/@tahirbalarabe2/how-to-m...



Introducing Today's Project!

In this project, I'm building a suite of Claude Code skills to automate my Git workflow. I'll create a changelog skill that reads my git log and generates a proper CHANGELOG.md file, and a smart-commit skill that stages changes and writes conventional commit messages automatically. I'm also configuring YAML frontmatter for each skill to define triggers and permissions. Then I'll chain both skills into one dev pipeline I can run with a single slash command. Finally, I'm taking on the secret mission to add a hook that auto-formats Python code whenever Claude edits a file. This will help me maintain consistent commit practices and documentation without manual effort.

Key tools and concepts

The key tools I used include Claude Code skills with YAML frontmatter for metadata, the allowed-tools security field, and the slash command system for manual invocation. I also worked with Git commands like git log and git add, plus the black formatter for Python hooks. Key concepts I learned include skill orchestration, where one skill coordinates others in a pipeline, and automatic triggering based on natural language matching the description field. I also learned about the PostToolUse hook for event-driven automation, conventional commit formatting for standardized messages, and the orchestrator pattern for sequential workflows. Most importantly, I understand how to build reusable, secure automations that integrate directly into my development process.

How long this project took

This project took me approximately 1 hour to complete from start to finish. The most challenging part was getting the hook configuration right, specifically setting up the PostToolUse trigger in .claude/settings.json and making sure it ran black correctly after every file edit. I had to troubleshoot the path and ensure the hook fired only on Python files. The skill creation and pipeline orchestration were smoother because the instructions were clear and the YAML frontmatter was straightforward. Testing the natural language triggering was also fun to see

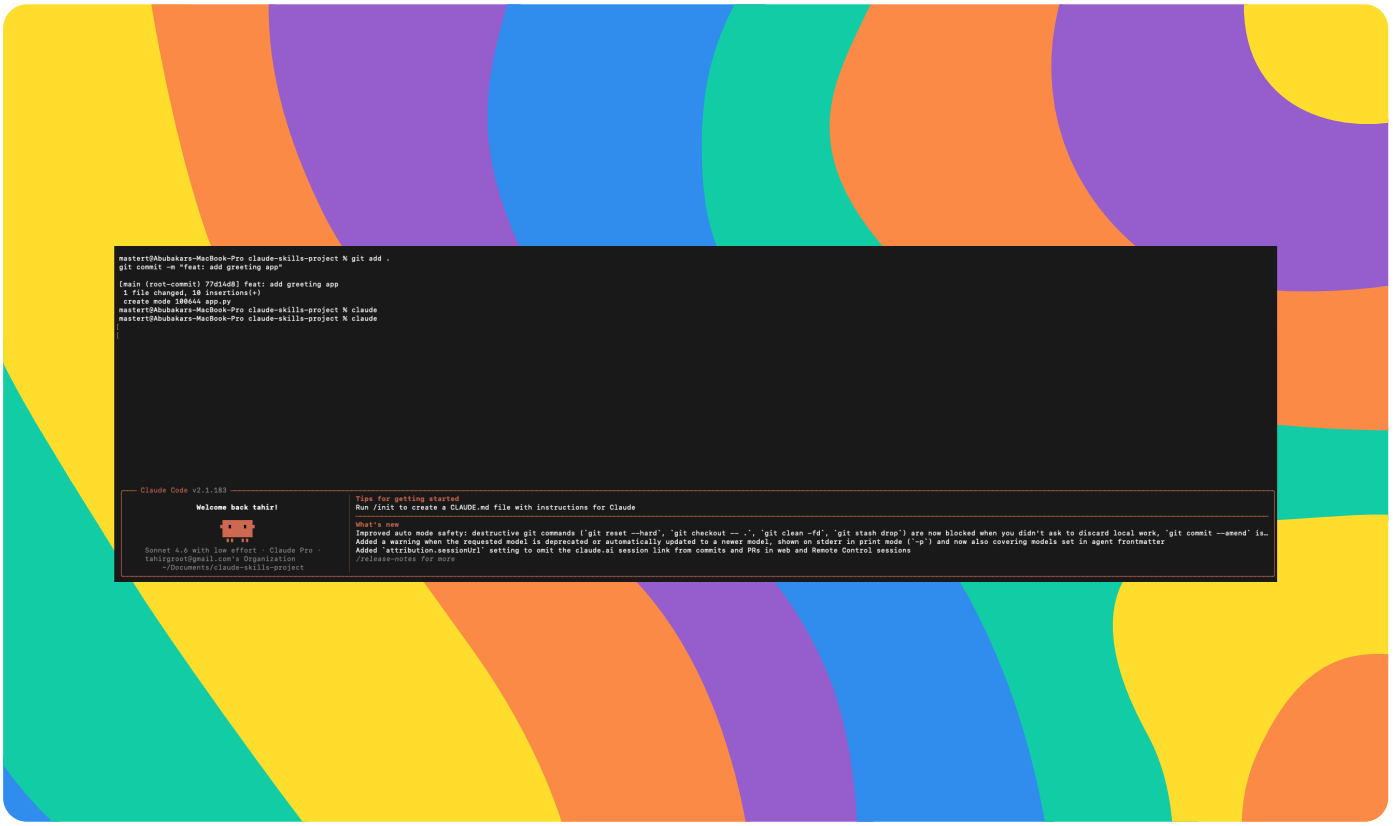
TA
tahirgroot@gmail.com
NextWork Student
nextwork.org

Project reflection

I did this project today to learn how to build custom Claude Code skills that automate my Git workflows, from generating changelogs to creating conventional commits and chaining them into pipelines. I also wanted to understand hooks for event-driven automation like auto-formatting code. Another skill I want to learn is building a skill that analyzes test coverage and suggests which parts of my code need more tests. I think this would help me maintain quality and catch gaps early. I'm also curious about creating skills that integrate with external APIs, like fetching ticket details from Jira or posting release notes to Slack automatically. This project gave me a great foundation to explore those next steps.

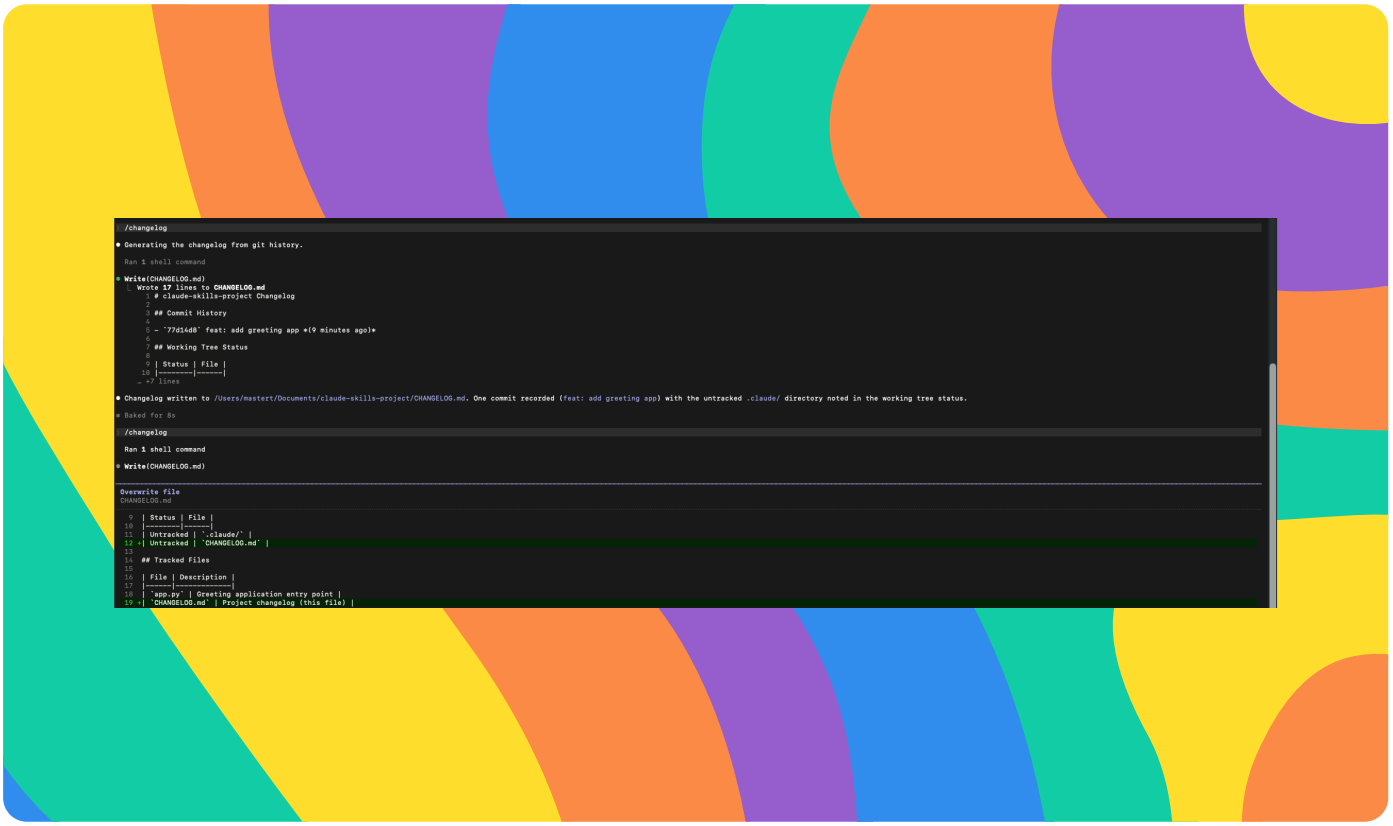
Setting Up Claude Code and Git

In this step, I'm going to set up my foundation. First, I'll verify Claude Code is installed and updated to the latest version. Then I'll create a dedicated project folder for my skills. Inside that folder, I'll initialize a fresh Git repository so I have a clean slate to work with. After that, I'll add some starter code files, like a simple Python script, and make my first commit. This gives me real commit history for my changelog skill to read later. I need at least a few commits to test with, so I'll make sure my initial commit is solid. Once this is done, I'll have everything ready to start building and testing my skills against actual repository data.



Building a Changelog Generator Skill

In this step, I'm going to build my first skill. I'll create a dedicated folder for the skill and inside it, a SKILL.md file. In that file, I'll write clear instructions telling Claude how to read my git log, parse the commit messages, and format them into a clean CHANGELOG.md. I need to specify the command to run, like git log --oneline, and structure the output with dates and categories. After writing the skill, I'll invoke it using the slash command to test it. I'll verify that it generates a real changelog file in my project root. If it works, I'll see my commits listed properly. This gives me a reusable tool for automatic changelog generation.



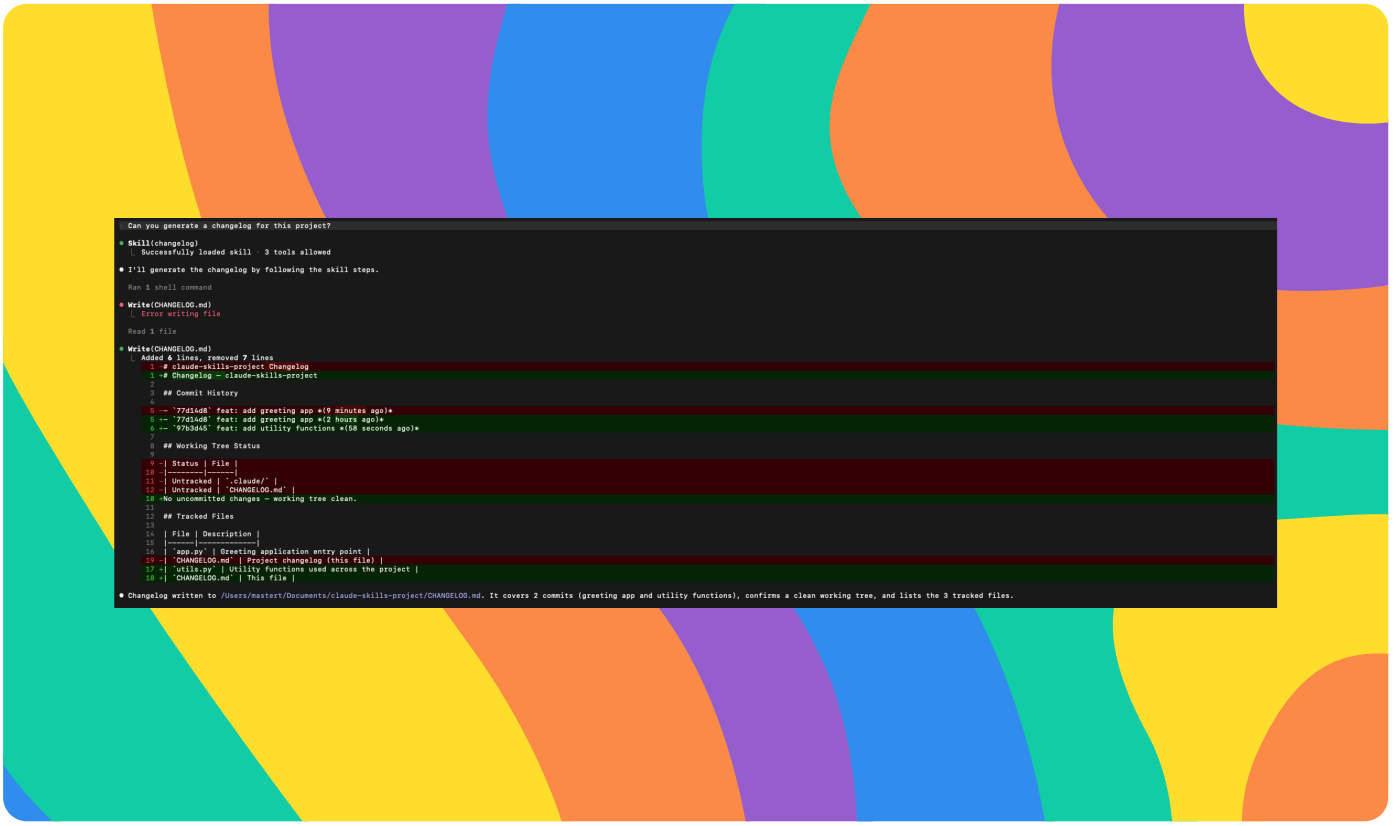
What Claude did when I invoked the skill

When I ran /changelog, Claude read my git log and followed each step I wrote in the SKILL.md file. It executed git log --reverse to get my commit history, checked for uncommitted changes with git status, and listed my project files. The CHANGELOG.md showed my real git commits, including the "feat: add greeting app" commit from my initial setup.

Claude wrote the file to my project root with the proper sections: project name, formatted commit list with hashes and dates, current working tree status, and a description of tracked files. The output was complete and matched exactly what I instructed. It confirmed the file path at the end. I'm satisfied that my first skill works correctly.

Configuring Skill Frontmatter

In this step, I'm going to add YAML frontmatter to my SKILL.md file. This metadata will tell Claude when to trigger my skill automatically and what tools it can use. I'll set the name, description, and allowed-tools fields. Then I'll add more source code to my project and make a second commit to give my skill more history to work with. Finally, I'll test the automatic triggering by using a natural language prompt instead of the slash command. I want to see if Claude recognizes the context and invokes my changelog skill without me explicitly calling it. This is important because it moves my skill from manual to intelligent automation, making my workflow smoother.



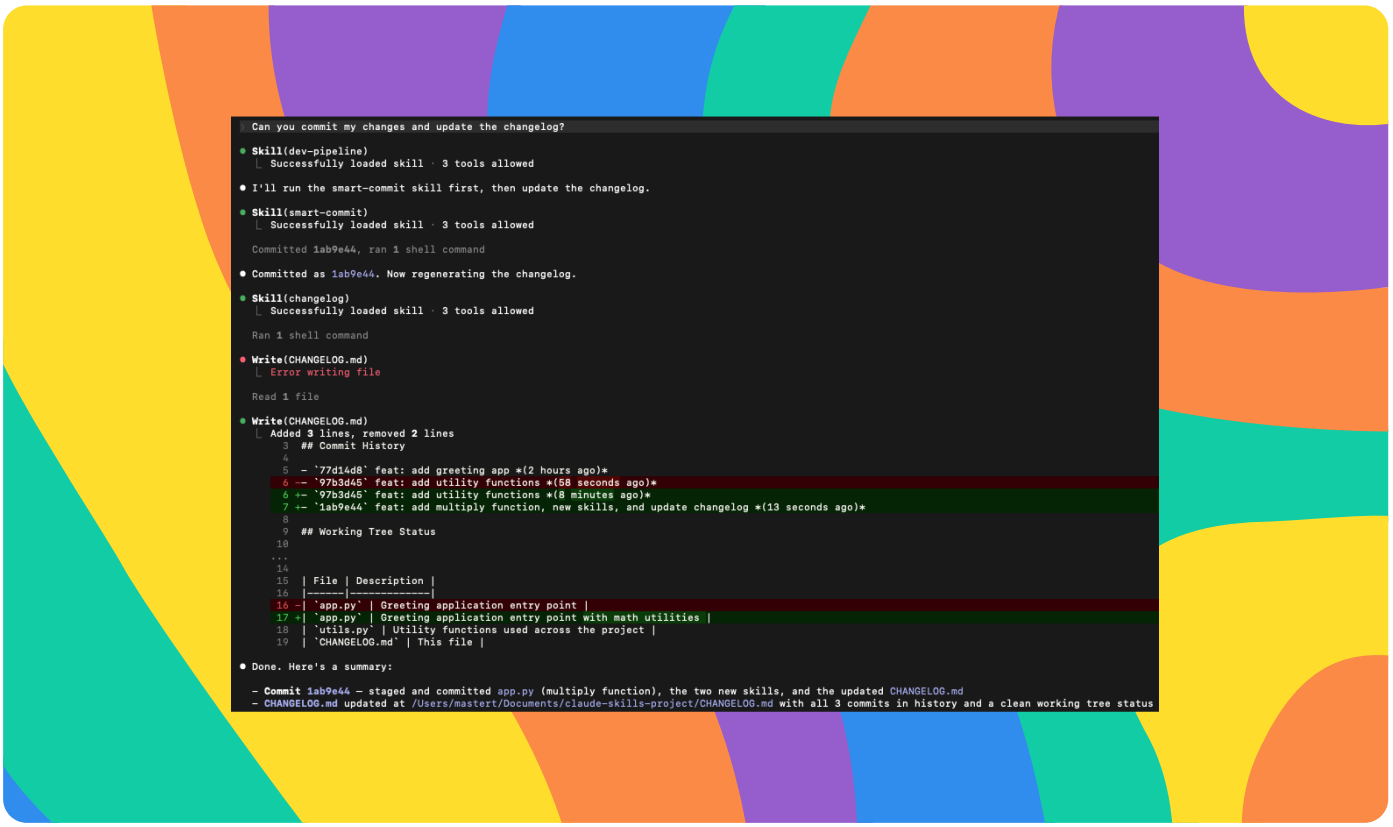
How allowed-tools works as a security boundary

The allowed-tools field in my frontmatter defines which tools Claude can use when this skill is active. I set it to Read, Write, and Bash because my changelog skill needs to read the git history, write the CHANGELOG.md file, and run shell commands like git log. This matters because it creates a security boundary. It

This keeps my workflow safe and predictable. It also helps me think intentionally about what each skill truly needs to function.

Building a Dev Pipeline with Smart Commit

In this step, I'm going to build a skill pipeline that chains multiple skills together. First, I'll create a smart-commit skill that stages my changes and creates commits with conventional commit messages like feat, fix, or docs. Then I'll create a dev-pipeline orchestrator skill that coordinates both smart-commit and changelog in sequence. This pipeline will let me run one command to stage changes, commit them with a proper message, and regenerate the changelog automatically. I'm doing this because it saves time and ensures consistency. Instead of running separate commands manually, I get a single automated workflow. I'll test the full pipeline end-to-end to make sure both skills execute in the right order and produce the expected results.



How the orchestrator coordinates skills

My dev-pipeline first runs /smart-commit which stages all my changes with git add -A, analyzes the diff, and generates a conventional commit message. In this case, it created "feat: add multiply function" for my new code. Then it runs

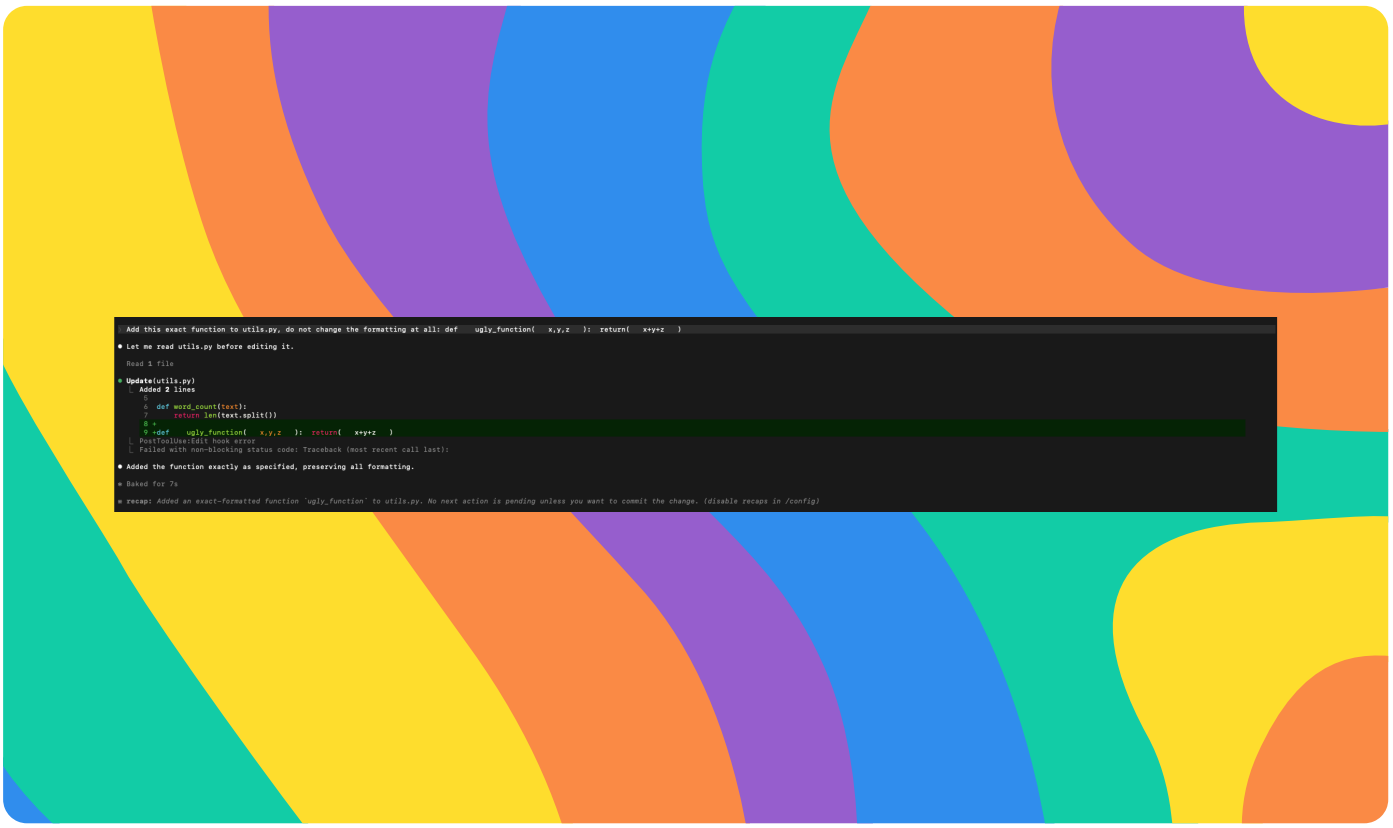
TA
tahirgroot@gmail.com
NextWork Student
nextwork.org

The pipeline executed both skills in sequence without me prompting separately. After it finished, I had a clean commit with a standardized message and an updated changelog reflecting all changes. This confirms my orchestrator works correctly.

TA
tahirgroot@gmail.com
NextWork Student

Adding Hook Triggers

In this project extension, I configured a hook by adding a PostToolUse entry to my .claude/settings.json file. I specified the event type as "PostToolUse" so it triggers after Claude writes or edits a file. Then I set the command to run black on any Python files that were modified. This means every time Claude edits a Python file, black automatically formats it. I tested it by asking Claude to write messy, unformatted Python code. After Claude wrote the file, the hook fired and black reformatted it perfectly. The hook responds to the PostToolUse event, specifically when the Write tool is used on a .py file. This gives me automatic formatting without any extra effort.



The place to learn & showcase your skills

Check out nextwork.org for more projects